

5G核心网测试用例管理系统

项目概述

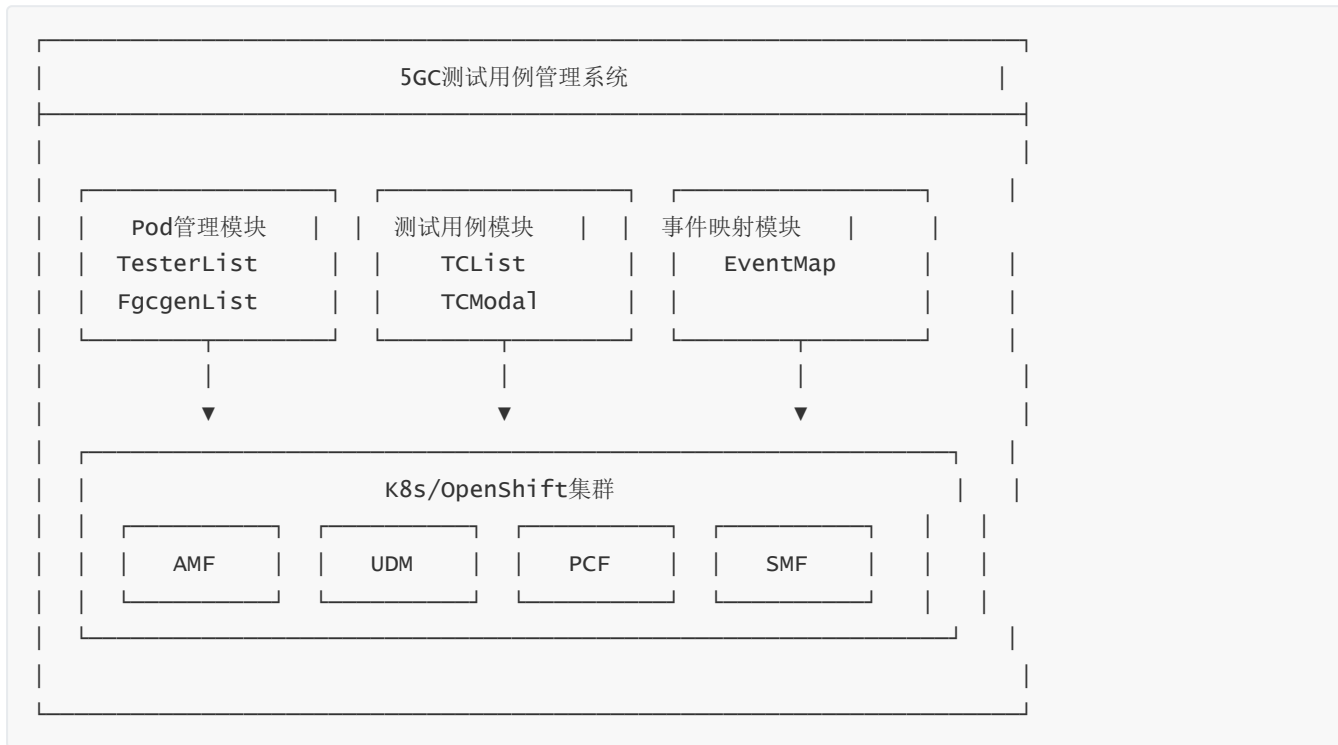
一、项目背景

在5G核心网（5GC）的开发与测试过程中，测试工程师需要对大量的**网络功能（NF）**进行配置和验证。传统的手工配置方式效率低下、容易出错，且难以管理复杂的测试场景。本项目旨在构建一个**可视化的测试用例管理系统**，通过Web界面实现对5G核心网测试环境的统一管理和自动化测试。

二、核心定位

属性	说明
项目名称	TCS Management (Test Case System Management)
产品定位	5G核心网SMF（Session Management Function）测试工具管理界面
目标用户	5G网络测试工程师、开发工程师
部署环境	Docker容器化 → OpenShift/K8s集群（内网部署）
访问方式	浏览器访问，Hash路由模式

三、核心功能模块



模块1：Pod管理模块（TesterList / FgcgenList）

功能	说明
Pod列表	展示K8s命名空间下的Tester/Fgcgen Pod状态
部署Pod	上传YAML文件部署新的测试Pod
删除Pod	删除指定Pod并释放资源
实时状态	500ms轮询自动刷新Pod状态
日志查看	实时流式传输Pod日志（ReadableStream）

模块2：测试用例模块（TCList / TCModal）★核心模块

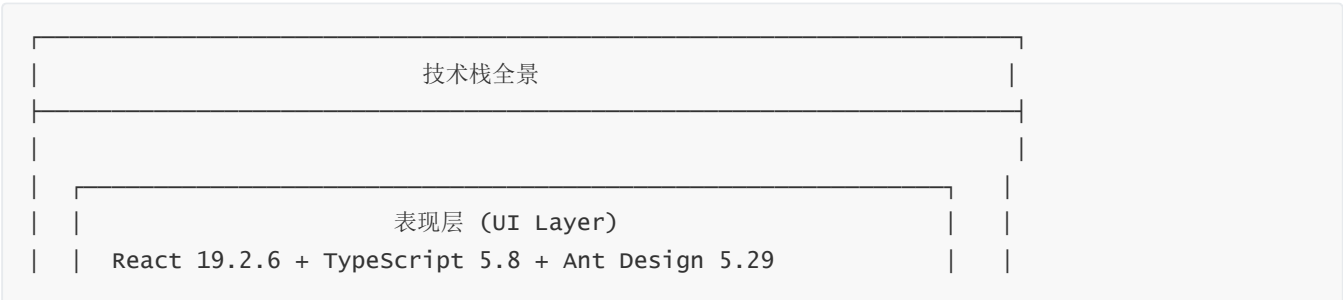
功能	说明
目录树导航	左侧树形结构展示测试用例分组
测试用例CRUD	新增、编辑、复制、删除、导出测试用例
动态NF配置	后端驱动的表单DSL，支持7种5G网元配置
文件管理	JSON/Shell文件的查看、编辑、上传、下载
批量执行	选择多个测试用例批量运行
执行摘要	实时查看测试用例执行状态和结果
日志查看	查看测试用例执行日志
Pcap下载	下载抓包文件进行分析

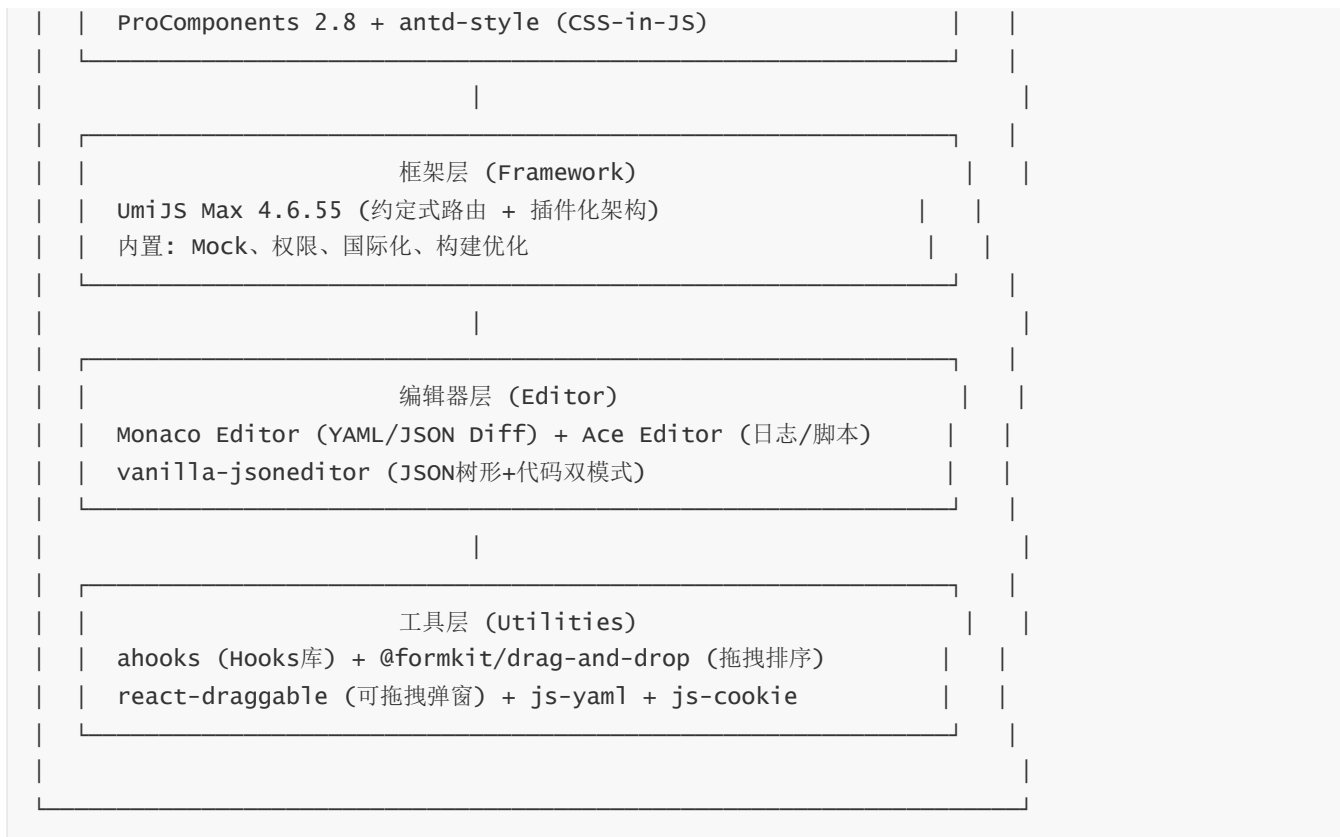
模块3：事件映射模块（EventMap）

功能	说明
PCAP转换	上传PCAP文件转换为结构化JSON事件
事件查看	在JSON编辑器中查看转换结果

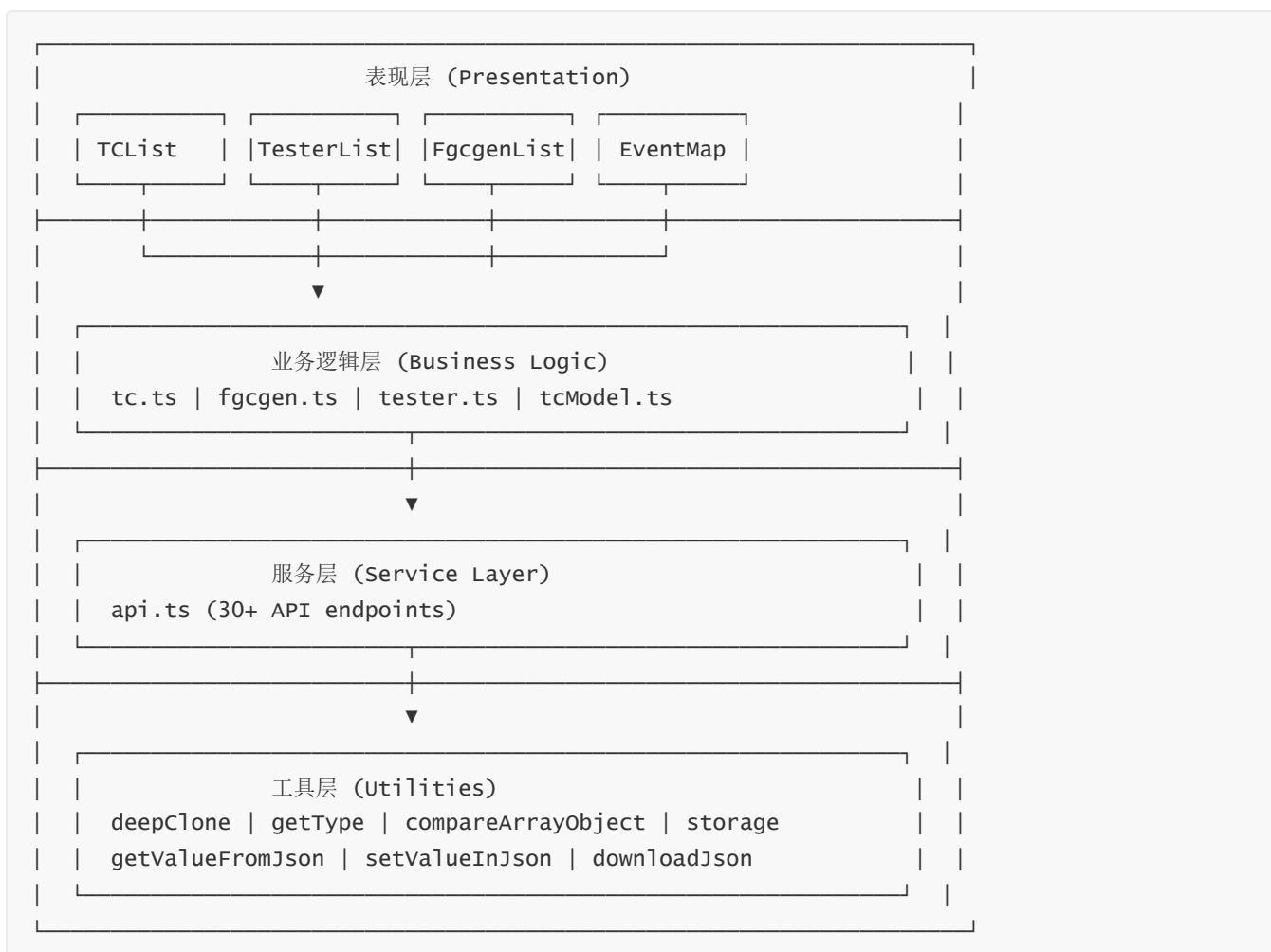
四、技术架构

4.1 技术栈全景





4.2 分层架构设计



五、项目规模

维度	数量	说明
页面组件	4个	TCList、TesterList、FgcgenList、EventMap
共享组件	7个	Editor、JsonEditor、YAMLEditor等
业务子组件	12个	TCModel下的所有子组件
最大组件	628行	EditTable.tsx（可编辑树表格）
API接口	30+	覆盖所有业务场景
Mock文件	14个	开发环境独立运行
工具函数	7个	deepClone、getType等
第三方依赖	20+	React生态核心库

六、核心数据结构

测试用例（TC）结构

```
interface TCCase {
  tcId: string;           // TC_ID格式：TC_X.X.X（2-6级）
  name: string;           // 测试用例名称
  description: string;    // 描述
  groupId: string;        // 所属分组
  config: {               // NF配置
    global: GlobalConfig; // 全局配置
    nf: {                 // 5G网元配置
      amf: NFConfig;
      udm: NFConfig;
      pcf: NFConfig;
      chf: NFConfig;
      nrf: NFConfig;
      upf: NFConfig;
      sgw: NFConfig;
    };
  };
  fileList: FileItem[];  // 关联文件列表
  status: string;         // 状态
  result: string;         // 执行结果
}
```

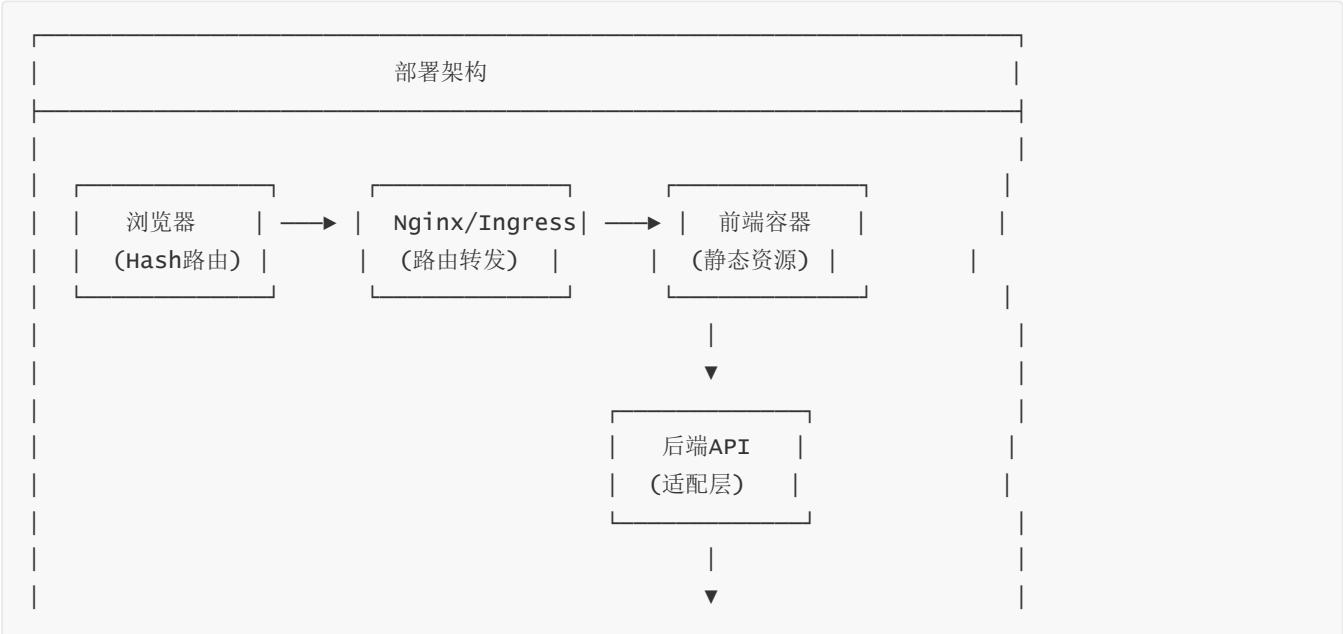
事件树结构

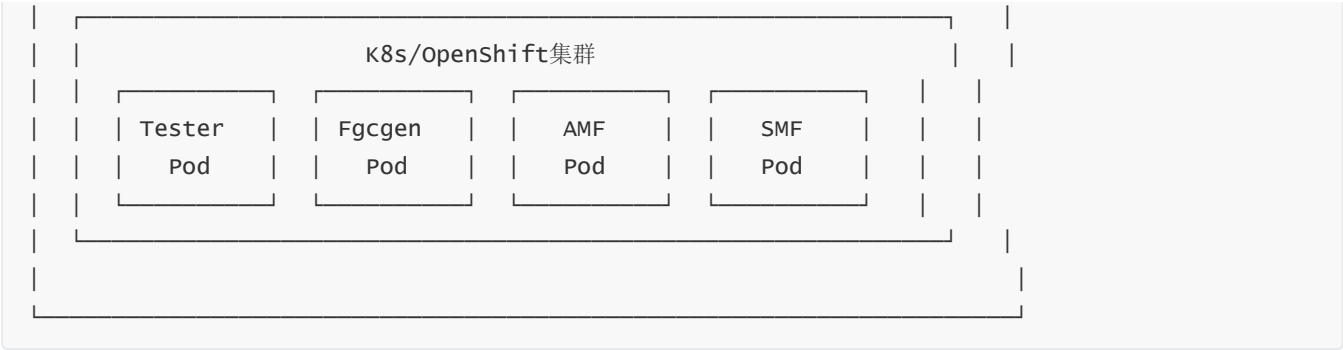
```
interface treeItem {
  key: string;           // 唯一标识 (格式: {type}-{randomId}-{path})
  title?: string;        // 显示标题
  require?: boolean;     // 是否必填
  isLeaf?: boolean;      // 是否叶子节点
  options?: string[];    // 下拉选项
  type: string;          // 控件类型 (text/float/digit/select/bool/map等)
  min?: number;          // 最小值
  max?: number;          // 最大值
  mode?: 'multiple' | 'tags' | 'single'; // 选择模式
  value?: any;           // 当前值
  children?: treeItem[]; // 子节点 (递归结构)
}
```

七、技术亮点速览

亮点	技术价值	难度
动态表单DSL引擎	后端驱动的表单配置，零代码适配新网元	☆☆☆
树形数据操作引擎	完整的递归CRUD操作库	☆☆☆
React 18并发特性	useDeferredValue + startTransition实战	☆☆☆
三模式文件编辑器	JSON/Events/Diff无缝切换	☆☆
流式日志传输	ReadableStream实时读取	☆☆
多编辑器集成	Ace + Monaco + vanilla-jsoneditor	☆☆
可拖拽模态框	突破Ant Design限制	☆

八、部署架构





九、CI/CD流程

代码提交 → GitLab CI触发 → 代码检查(ESLint/TypeScript) → 构建打包 → Docker镜像构建 → 推送镜像仓库 → 部署到K8s

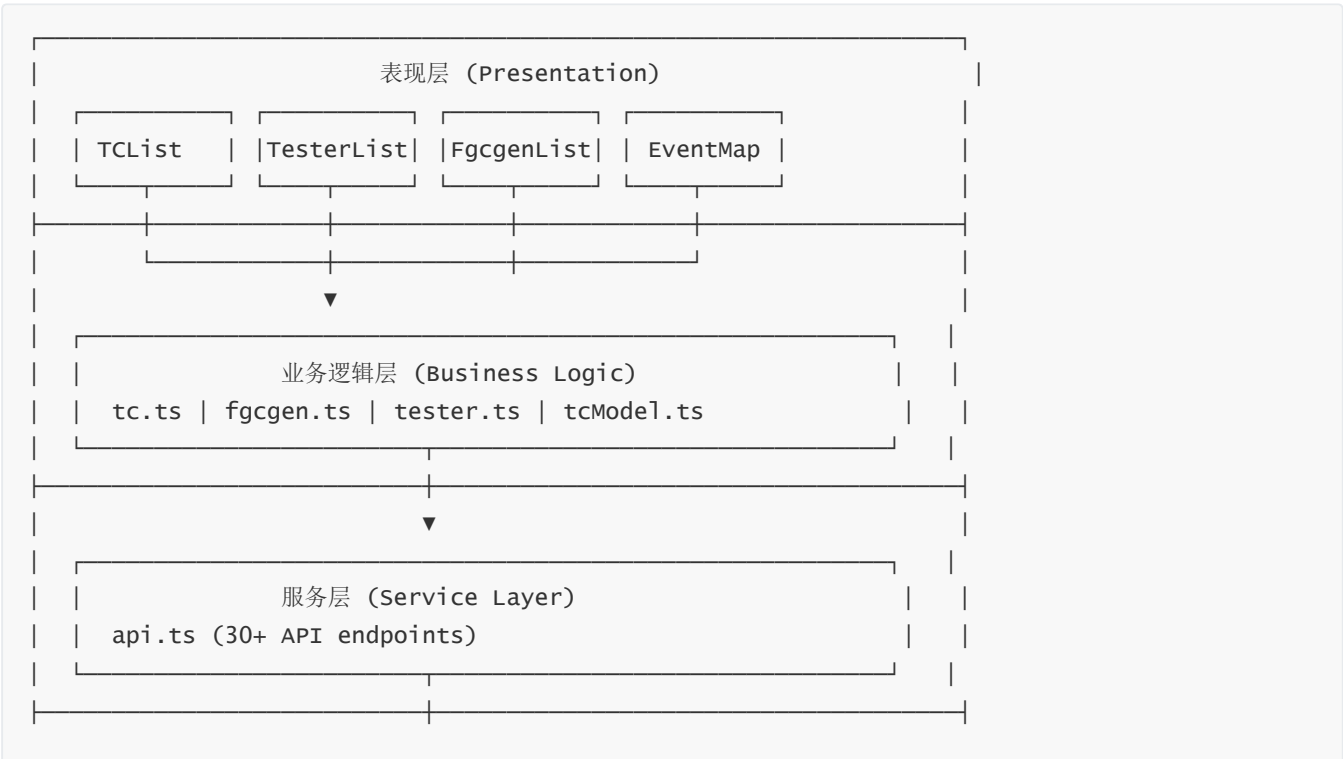
十、面试价值总结

本项目具有以下面试讲述价值：

- 1. **架构设计能力**：动态表单DSL引擎、分层架构设计、状态管理策略
- 2. **算法设计能力**：树形数据递归操作、Key唯一性保证算法
- 3. **React深度应用**：React 18并发特性、性能优化策略
- 4. **工程化能力**：CI/CD、多环境配置、代码质量保障
- 5. **问题解决能力**：可拖拽模态框、流式日志传输、多编辑器集成

一、项目架构全景

1.1 分层架构设计



工具层 (Utilities)

deepClone | getType | compareArrayObject | storage
getValueFromJson | setValueInJson | downloadJson

1.2 核心模块依赖关系

config/routes.ts

```
└─ TList/index.tsx (主页面 - 左右分栏布局)
  │   └─ tc.ts (业务逻辑层)
  │       └─ api.ts (API层)
  │   └─ RightTree.tsx (目录树导航)
  │   └─ TCModal (可拖拽多标签模态框)
  │       └─ updateConfig.tsx (动态NF配置面板)
  │           └─ RenderForm.tsx (递归动态表单引擎)
  │               └─ getType (工具函数)
  │           └─ FileList.tsx (文件列表管理)
  │               └─ FileModal.tsx (三模式JSON编辑器)
  │                   └─ Events.tsx (事件拖拽表格)
  │                       └─ EditTable.tsx (可编辑树表格 - 628行)
  │                           └─ edit.ts (树数据操作引擎)
  │                           └─ AddEvent.tsx (新增事件)
  │                           └─ JsonEditor.tsx (共享组件)
  │                           └─ ShellModal.tsx (Shell脚本编辑)
  │           └─ tcModel.ts (模态框业务逻辑)
  │   └─ TCSummary.tsx (测试执行摘要)
  │   └─ Editor.tsx (日志查看 - Ace)
  └─ JsonEditor.tsx (TC查看 - vanilla-jsoneditor)
```

二、技术难点深度剖析

2.1 动态表单DSL渲染引擎 ★★ ★

位置: src/pages/TList/TCModel/components/RenderForm.tsx (362行)

难点分析

这是一个**后端驱动的表单DSL系统**，前端需要将后端返回的JSON Schema递归解析并渲染为完整的Ant Design表单。

核心挑战:

- 无限层级嵌套:** tabs → tab → card → form → leaf fields, 支持任意深度
- 6种叶子字段类型:** string、int、float、select、intArray、floatArray
- 动态路径映射:** 每个表单项的 name 需要根据嵌套层级动态构建
- 数组类型支持:** ProFormList实现动态增删, 每项包装在可折叠ProCard中

设计方案

```
// 三函数递归架构
renderForm(item, path, currentTab)    // 顶层分发器: tabs/card/form
└─> renderFormItem(data, path, currentTab) // 处理Object/Array嵌套
      └─> renderData(data, path, currentTab, type) // 渲染叶子节点
            └─> renderChildren(children, path, currentTab) // 递归子项
```

类型到控件映射表:

controlType	渲染组件	特殊处理
string	ProFormText	hasFeedback实时校验
int	ProFormDigit	precision: 0, min/max安全整数
float	ProFormDigit	precision: 10, 无步进器
select	ProFormSelect	支持boolean选项转换
intArray	ProFormText	正则校验 + transform数组转换
floatArray	ProFormText	正则校验 + transform数组转换
tabs	Tabs + ProCard	递归渲染tab子项
card	ProCard	可折叠, 递归渲染children
form	递归renderFormItem	支持Object/Array两种模式

数组类型特殊处理:

```
// intArray/floatArray的双向转换
convertValue={value => Array.isArray(value) ? value?.join(',') : value}
transform={value => {
  const tmp = value?.split(',').map((item: any) => item ? Number(item) : '');
  return { [data[item].key]: { value: tmp?.filter((i: any) => i !== '') } } };
}}
```

2.2 树形数据操作引擎 ★★

位置: src/pages/TCList/TCModel/components/edit.ts (209行)

难点分析

事件树是一个可编辑的嵌套结构, 需要支持:

- 1. 递归增删改查: 任意层级节点的CRUD操作
- 2. Key唯一性保证: 复制/添加时重新生成所有子孙节点的Key
- 3. 操作权限控制: 不同类型节点 (select/map) 有不同的操作权限

4. 递归校验: 遍历所有节点收集校验失败的Key

核心算法

Key重新生成算法:

```
export const updateKeysForAddOrCopy = (
  data: treeItem[],
  type: 'ADD' | 'COPY',
  parentKey = ''
) => {
  data?.forEach((item: treeItem) => {
    if (item?.key) {
      // 保留原始Key的最后一段作为标识
      const currentKeys = item?.key?.split('-');
      item.key = newKey + currentKeys.pop();

      // ADD模式清空值, COPY模式保留值
      if (type === 'ADD') delete item?.value;
    }

    // 子节点使用随机数生成唯一Key
    if (item?.children && Array.isArray(item?.children)) {
      newKey = parentKey
        ? `${parentKey}-${(Math.random() * 1000000).toFixed(0)}`
        : item?.key;
      updateKeysForAddOrCopy(item?.children, type, newKey);
    }
  });
  return [{ ...data[0], key: parentKey }];
};
```

操作权限Map生成:

```
export const generateItemToKey = (data: treeItem[], map = new Map()) => {
  for (const item of data) {
    if (item.type === 'select') {
      // select类型: 子节点可复制, 多于1个时可删除
      item.children?.forEach((ch) => map.set(ch.key + 'copy', true));
      if (item.children?.length > 1) {
        item.children.forEach((ch) => map.set(ch.key + 'delete', true));
      }
    } else if (item.type === 'map') {
      // map类型: 子节点可复制、可删除、可添加map
      item.children?.forEach((ch) => {
        map.set(ch.key + 'map', true);
        map.set(ch.key + 'copy', true);
      });
      if (item.children?.length > 1) {
        item.children.forEach((ch) => map.set(ch.key + 'delete', true));
      }
    }
  }
}
```

```
// 递归处理子节点
if (item?.children) generateItemToKey(item?.children, map);
}
return map;
};
```

递归校验算法:

```
export const validateAllKeys = (data: treeItem[]) => {
  const typeLists = ['text', 'float', 'select', 'digit', 'bool',
    'digitArray', 'floatArray', 'textArray'];
  let keys: string[] = [];

  data?.forEach((item) => {
    if (!item?.children && item.require && typeLists.includes(item?.type)) {
      // 校验空值
      if (!isEmpty(item?.value)) {
        keys.push(item?.key);
      } else {
        // 校验格式规则
        if (item?.type === 'digitArray') {
          const expr = /^d+(?:[,d+])?$/ .test(item?.value);
          if (!expr) keys.push(item?.key);
        } else if (item?.type === 'floatArray') {
          const expr = /^(-?d+(\.d+)?)(-?d+(\.d+)?)*/ .test(item?.value);
          if (!expr) keys.push(item?.key);
        }
      }
    }
    if (item?.children) {
      keys = keys.concat(validateAllKeys(item?.children));
    }
  });
  return keys;
};
```

2.3 实时日志流式传输 ★ ★

位置: `src/pages/FgcgenList/index.tsx`

难点分析

需要实现**实时日志流**，类似 `tail -f` 的效果，传统轮询方案延迟高、资源浪费。

技术方案

```
// 1. 创建AbortController支持取消
const abortController = new AbortController();
const signal = abortController.signal;

// 2. 递归读取chunk
```

```

const processChunks = async (reader: ReadableStreamDefaultReader<Uint8Array>) => {
  try {
    const { done, value } = await reader.read();
    if (done) return;

    const chunk = new TextDecoder('utf-8').decode(value);
    setLogData((prev) => prev + chunk);

    await processChunks(reader); // 递归读取下一块
  } catch (err: unknown) {
    if ((err as Error).name !== 'AbortError') {
      console.error('processChunks error', err);
    }
  }
};

// 3. 发起流式请求
const fetchChunkedData = async () => {
  const response = await fetch(url, {
    method: 'GET',
    signal,
    headers: { Accept: 'text/plain; chunks=true' },
  });

  const reader = response.body?.getReader();
  if (reader) await processChunks(reader);
};

// 4. 组件卸载时清理
useEffect(() => {
  return () => {
    abortController.abort();
    setLogData('');
  };
}, [logModalOpen, fgcbgen]);

```

关键点:

- 使用 `ReadableStream` 分块读取，避免一次性加载大文件
- `AbortController` 支持请求取消，防止内存泄漏
- 递归读取避免回调地狱，代码更清晰

2.4 可编辑事件树表格 ★★ ★

位置: `src/pages/TCList/TCModel/components/EditTable.tsx` (628行 - 项目最大组件)

难点分析

这是整个项目中**最复杂的组件**，需要实现：

1. **EditableProTable + 树形展开**: 行内编辑 + 树形子节点
2. **React 18性能优化**: useDeferredValue + startTransition
3. **防抖校验**: 500ms防抖收集校验失败的Key
4. **DOM操作增强**: 直接操作DOM设置错误边框高亮

React 18新特性应用

```
// 1. useDeferredValue - 延迟更新避免卡顿
const tree = useMemo(() => recordList, [JSON.stringify(recordList)]);
const deferredQuery = useDeferredValue(tree);

// 2. startTransition - 标记非紧急更新
const handleSave = (record: treeItem, value: any) => {
  startTransition(() => {
    setRecordList(newData); // 标记为非紧急更新
  });
};

// 3. useDebounceFn - 防抖校验
const { run: debouncedValidate } = useDebounceFn(
  (data: treeItem[]) => {
    const unValidateKeys = validateAllKeys(data);
    unValidateKeysRef.current = unValidateKeys;
    // DOM操作设置错误边框
    setCardBorderColor(unValidateKeys);
  },
  { wait: 500 }
);
```

2.5 三模式文件编辑器 ★★

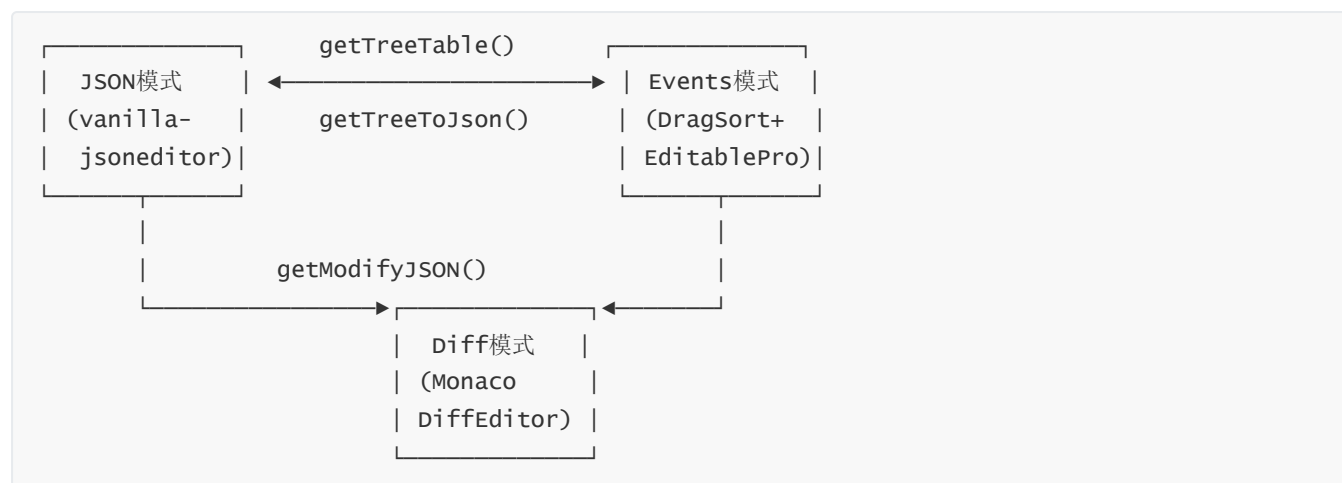
位置: `src/pages/TCList/TCTModel/components/FileModal.tsx` (536行)

难点分析

需要支持**三种编辑模式**的无缝切换：

1. **JSON模式**: vanilla-jsoneditor (树形+代码双视图)
2. **Events模式**: DragSortTable + EditableProTable (可视化编辑)
3. **Diff模式**: Monaco DiffEditor (原始vs修改后对比)

数据流设计



关键设计:

- 所有转换调用**后端API**完成，前端不做转换逻辑
- 保证数据一致性，避免前后端转换逻辑不同步
- `startTransition` 优化模式切换时的UI响应性

2.6 可拖拽模态框 ★ ★

位置: `src/pages/TCList/TCModel/index.tsx`

难点分析

Ant Design Modal**不支持拖拽**，需要突破框架限制。

实现方案

```
// 1. 边界约束计算
const bounds = {
  left: 0,
  top: 0,
  right: window.document.documentElement.clientWidth - draggableRef.current?.offsetWidth,
  bottom: window.document.documentElement.clientHeight - draggableRef.current?.offsetHeight,
};

// 2. 仅标题区域可拖拽
<div
  onMouseOver={() => setDisabled(false)}
  onMouseOut={() => setDisabled(true)}
>
  {title}
</div>

// 3. react-draggable集成
<Draggable disabled={disabled} bounds={bounds} nodeRef={draggableRef}>
  <div ref={draggableRef}>{modal}</div>
</Draggable>
```

三、设计模式与架构亮点

3.1 设计模式应用

模式	应用场景	实现
Presenter/Logic分离	tc.ts、fgcgen.ts、tester.ts	业务逻辑与UI组件分离
递归渲染模式	RenderForm.tsx	JSON Schema → React组件
策略模式	ErrorShowType枚举	按错误类型选择不同处理策略
观察者模式	Form.useWatch	监听表单字段变化
组合模式	树形数据结构	统一处理叶子节点和容器节点
代理模式	api.ts统一前缀	环境适配的API代理

3.2 状态管理策略

轻量级、本地优先的架构设计：

状态管理策略

1. UmiJS initialState (全局)

↳ 用户信息、布局配置

2. UmiJS Model Plugin (轻量全局)

↳ dataSource状态 (src/models/configuration.ts)

3. React Hooks (组件级 - 主要方式)

↳ useState: UI状态、表单数据

↳ useRef: 不触发重渲染的持久化数据

↳ useMemo: 缓存计算结果

↳ useDeferredValue: 延迟更新

4. localStorage (持久化)

↳ 展开状态、用户偏好

✗ 不使用Redux/MobX/Zustand

原因: 项目复杂度适中, hooks足够

3.3 错误处理体系

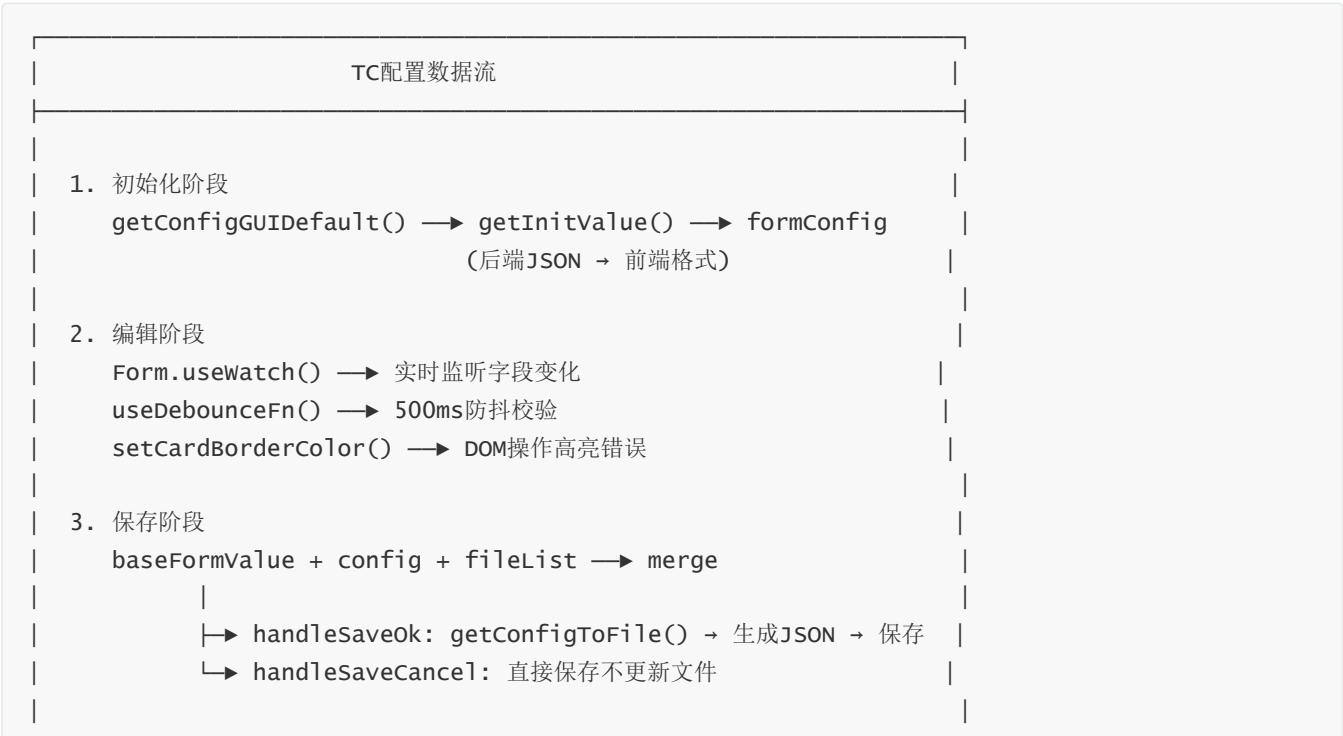
统一的五级错误处理:

```
enum ErrorShowType {
  SILENT = 0,          // 静默
  WARN_MESSAGE = 1,    // 警告弹窗
  ERROR_MESSAGE = 2,   // 错误弹窗
  NOTIFICATION = 3,    // 通知框
  REDIRECT = 9,        // 重定向
}

// 错误分流处理
const errorHandler = (error: ResponseError) => {
  if (error.name === 'BizError') {
    // 业务错误: 按showType选择UI展示
    const errorInfo = error.info;
    if (errorInfo.errorCode === 'ERROR_1') {
      message.warning(errorInfo.message);
    }
  } else if (error.response?.status) {
    // HTTP错误: 显示状态码
    message.error(`请求错误 ${error.response.status}`);
  } else {
    // 网络错误: 提示重试
    message.error('网络异常, 请检查网络连接');
  }
};
```

3.4 数据流设计

TC配置的数据流:



四、React 18新特性实战应用

4.1 useDeferredValue

应用场景: EditTable组件的树形数据更新

```
// 延迟更新，避免编辑时卡顿
const tree = useMemo(() => recordList, [JSON.stringify(recordList)]);
const deferredQuery = useDeferredValue(tree);

// 使用deferredQuery渲染表格
<EditableProTable value={deferredQuery} />
```

原理:

- `useDeferredValue` 会创建一个延迟版本的state
- 高优先级更新（用户输入）会打断低优先级更新（表格渲染）
- 避免复杂表格渲染阻塞用户交互

4.2 startTransition

应用场景: FileModal模式切换、EditTable数据更新

```
// 标记为非紧急更新
const handleModeChange = (mode: 'json' | 'events' | 'diff') => {
  startTransition(() => {
    setMode(mode); // 模式切换
  });
};

// 表单数据更新
const handleSave = (record: treeItem, value: any) => {
  startTransition(() => {
    setRecordList(newData); // 树数据更新
  });
};
```

原理:

- `startTransition` 告诉React当前更新是"过渡性的"
- React可以中断这个更新去处理更紧急的任务（如用户输入）
- 避免UI卡顿，提升用户体验

4.3 useDebounceFn

应用场景: EditTable校验、表单搜索

```
// ahooks的防抖函数
const { run: debouncedValidate } = useDebounceFn(
  (data: treeItem[]) => {
    const unvalidateKeys = validateAllKeys(data);
    setCardBorderColor(unvalidateKeys);
  },
  { wait: 500 }
);

// AutoComplete搜索
const { run: debouncedSearch } = useDebounceFn(
  (value: string) => {
    setSearchResult(filterOptions(value));
  },
  { wait: 300 }
);
```

五、性能优化策略

5.1 组件级优化

React.memo + 自定义比较

```
// 5个核心组件使用memo优化
const RightTree = memo(({ data, ...props }) => {
  // ...
}, (oldProps, newProps) =>
  JSON.stringify(oldProps.data) === JSON.stringify(newProps.data)
);

// 应用组件
- RightTree (目录树导航)
- UpdateConfig (动态配置面板)
- EditTable (可编辑树表格)
- Events (事件列表)
- RenderForm (动态表单)
```

useRef避免重渲染

```
// eventRef存储事件数据，修改不触发重渲染
const eventRef = useRef<DataSourceType[]>([]);

// 折叠时同步数据，不丢失编辑状态
const handleExpand = (expanded: boolean, record: treeItem) => {
  if (!expanded) {
    // 折叠时将ref数据同步到state
    setEventLists(eventRef.current);
  }
};
```

5.2 数据级优化

useMemo缓存计算

```
// 缓存树扁平化结果
const flatTreeData = useMemo(() => {
  return flattenTree(treeData);
}, [treeData]);

// 缓存筛选后的选项
const filteredOptions = useMemo(() => {
  return options.filter(opt =>
    opt.label.toLowerCase().includes(searchValue.toLowerCase())
  );
}, [options, searchValue]);
```

5.3 网络级优化

轮询控制

```
const [polling, setPolling] = useState(500); // 500ms轮询

// 用户可手动切换
<Button onClick={() => setPolling(prev => prev ? 0 : 500)}>
  {polling ? '停止轮询' : '开启轮询'}
</Button>

// ProTable内置轮询
<ProTable polling={polling} />
```

5.4 构建级优化

```
// config/config.ts
export default defineConfig({
  // 1. MFSU - 模块联邦预构建加速
  mfsu: { mode: 'normal' },
```

```
// 2. esbuild压缩
jsMinifier: 'esbuild',
esbuildMinifyIIFE: true,

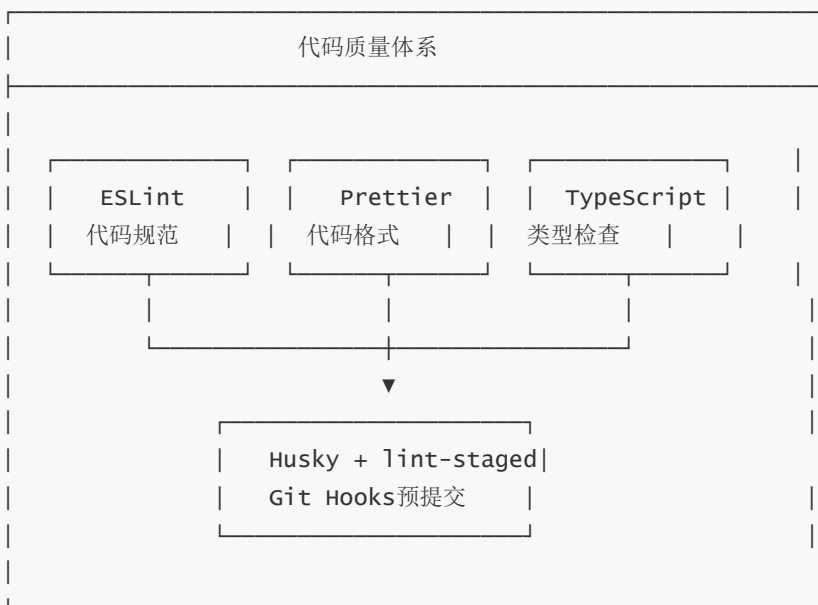
// 3. 生产环境移除console
drop: process.env.NODE_ENV === 'production'
  ? ['console', 'debugger']
  : [],

// 4. Monaco Editor按需加载
plugins: [new MonacoWebPackPlugin({ languages: ['json'] })],

// 5. dayjs替代moment
moment2dayjs: { preset: 'antd', plugins: ['duration'] },
});
```

六、工程化体系

6.1 代码质量保障



6.2 多环境配置

```
// 环境矩阵
const envConfig = {
  dev: {
    prefix: '/api/smf-tester/adapter',
    proxy: 'http://smf-tester-smf-upf-tester-1.apps.ocp134.casa-smf.site/',
    mock: true,
  },
  test: {
    prefix: '/api/smf-tester/adapter',
    proxy: null,
  },
};
```

```
    mock: false,
  },
  pre: {
    prefix: '/smf-tester/adapter',
    proxy: null,
    mock: false,
  },
  production: {
    prefix: '/smf-tester/adapter',
    proxy: null,
    mock: false,
  },
};
```

6.3 CI/CD流水线

```
# .gitlab-ci.yml
image: registry.gitlab.casa-systems.com/vmc-ui/nodejs:20-alpine3.18

stages:
  - install
  - lint
  - build
  - deploy

install:
  script:
    - npm ci
  cache:
    paths:
      - node_modules/

lint:
  script:
    - npm run lint:js
    - npm run tsc
  only:
    - merge_requests

build:
  script:
    - cross-env REACT_APP_ENV=production max build
  artifacts:
    paths:
      - dist/
  only:
    - main

deploy:
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

```
only:
  - main
```

6.4 国际化支持

```
// 7个国际化模块
src/locales/
└─ en-US/
    ├── component.ts      # 组件文本
    ├── globalHeader.ts   # 全局头部
    ├── menu.ts           # 菜单
    ├── pages.ts          # 页面
    ├── pwa.ts            # PWA
    ├── settingDrawer.ts  # 设置面板
    └── settings.ts       # 配置
```

七、组件设计亮点

7.1 多编辑器统一管理

编辑器	用途	特性	行数
Ace Editor	日志/脚本查看	轻量、只读、语法高亮	~50行
Monaco Editor	YAML编辑	K8s Schema提示、智能感知	~80行
vanilla-jsoneditor	JSON编辑	树形+代码双模式	~60行

统一组件封装:

```
// src/components/share/
├─ Editor.tsx           # Ace编辑器封装
├─ JsonEditor.tsx       # vanilla-jsoneditor封装
├─ YAMLEditor.tsx       # Monaco YAML编辑器封装
└─ types.d.ts          # 类型定义
```

7.2 动态NF配置面板

7种5G网元支持: AMF、UDM、PCF、CHF、NRF、UPF、SGW

```
// 根据global.simNF动态生成Tab
const nfTabs = [
  { key: 'amf', label: 'AMF' },
  { key: 'udm', label: 'UDM' },
  { key: 'pcf', label: 'PCF' },
  { key: 'chf', label: 'CHF' },
  { key: 'nrf', label: 'NRF' },
  { key: 'upf', label: 'UPF' },
  { key: 'sgw', label: 'SGW' },
]
```

```

];

// ProFormDependency监听simNF变化
<ProFormDependency name={['global', 'simNF']}>
  {[simNF]} => (
    <Tabs items={nfTabs.filter(tab => simNF?.includes(tab.key))} />
  )
</ProFormDependency>

```

7.3 搜索导航系统

AutoComplete + DOM操作实现表单搜索导航:

```

// 1. 扁平化所有表单字段路径
const pathMapToData = traverseNFConfig(formConfig);

// 2. AutoComplete搜索
<AutoComplete
  options={pathMapToData.map(p => ({ label: p.path, value: p.path })))
  onSelect={(value) => scrollToPath(value)}
/>

// 3. DOM操作滚动到指定字段
const scrollToPath = (path: string) => {
  // 激活对应Tab
  setActiveTab(path.split('.')[0]);

  // 滚动到指定元素
  setTimeout(() => {
    const element = document.querySelector(`[data-path="${path}"]`);
    element?.scrollIntoView({ behavior: 'smooth', block: 'center' });
  }, 100);
};

```

八、面试高频问题（深度版）

8.1 React核心概念

Q1: 为什么使用JSON.stringify进行props比较? 如何优化?

答:

```

// 当前实现
(oldProps, newProps) => JSON.stringify(oldProps.item) === JSON.stringify(newProps.item)

// 优点
- 简单直接, 适用于复杂嵌套对象
- 不需要引入额外依赖

// 缺点

```

- 性能开销：每次比较都需要序列化 $O(n)$
- 无法处理函数、`Symbol`、循环引用
- 属性顺序不同会导致比较失败

// 优化方案

1. 使用`lodash.isEqual`进行深度比较（推荐）
2. 使用`react-fast-compare`（专门为React优化）
3. 对于简单结构，使用浅比较+不可变数据

Q2: `useRef`和`useState`的本质区别？项目中如何应用？

答:

特性	<code>useState</code>	<code>useRef</code>
触发重渲染	✅ 是	❌ 否
值可变性	需要 <code>setState</code>	<code>.current</code> 直接修改
适用场景	需要UI更新的状态	不需要UI更新的持久化数据
记忆性	每次渲染都是新值	跨渲染保持引用

本项目应用:

```
// 1. eventRef - 存储事件数据，折叠时不丢失
const eventRef = useRef<DataSourceType[]>([]);

// 2. unValidateKeysRef - 存储校验失败的key
const unValidateKeysRef = useRef<string[]>([]);

// 3. draggleRef - DOM引用
const draggleRef = useRef<HTMLDivElement>(null);
```

Q3: React 18的`startTransition`和`useDeferredValue`有什么区别？

答:

`startTransition`:

- 标记整个状态更新为"过渡性的"
- React可以中断这个更新去处理更紧急的任务
- 适用于：用户触发的离散操作（点击、输入）

`useDeferredValue`:

- 创建一个延迟版本的state
- 高优先级更新会打断低优先级更新
- 适用于：连续输入（搜索、滚动）

本项目应用:

```
// startTransition - 模式切换
const handleModeChange = (mode) => {
  startTransition(() => {
    setMode(mode); // 非紧急更新
  });
};

// useDeferredValue - 树数据更新
const tree = useMemo(() => recordList, [JSON.stringify(recordList)]);
const deferredQuery = useDeferredValue(tree); // 延迟更新
```

8.2 架构设计

Q4: 如何设计一个后端驱动的动态表单系统？

答:

核心设计:

- 1. Schema定义层
 - ↳ 后端定义JSON Schema (controlType、label、校验规则)
- 2. 解析渲染层
 - ↳ 前端递归解析，根据controlType分发渲染
- 3. 路径管理层
 - ↳ 动态构建表单项name路径，支持嵌套
- 4. 数据转换层
 - ↳ 前端格式 ↔ 后端格式的双向转换

关键技术点:

- 递归渲染三函数架构 (renderForm → renderFormItem → renderData)
- 动态路径拼接 (支持任意层级嵌套)
- ProFormList数组支持 (动态增删)
- 表单校验与DOM高亮集成

Q5: 如何设计一个可扩展的树形数据操作引擎？

答:

核心设计原则:

- 1. 单一职责: 每个函数只做一件事
- 2. 递归统一: 所有操作都支持任意层级
- 3. Key唯一性: 复制/添加时重新生成所有子孙节点的Key

4. 权限控制: 不同类型节点有不同的操作权限

核心函数:

```
// CRUD操作
addChildren(data, parentKey, newRow)      // 递归添加
removeChildren(data, parentKey, removeRow) // 递归删除
updateKeysForAddOrCopy(data, type, parentKey) // key重生成

// 查询操作
generateItemToKey(data, map) // 生成操作权限Map
extractAllKeys(data)         // 提取所有叶子Key
validateAllKeys(data)        // 递归校验
```

Q6: Hash路由和History路由的区别? 为什么选择Hash?

答:

特性	Hash路由	History路由
URL格式	<code>/#/path</code>	<code>/path</code>
服务器配置	不需要	需要配置fallback
SEO	较差	较好
兼容性	更好	需要PushState支持
状态保持	URL不刷新	支持刷新保持状态

本项目选择Hash原因:

1. 部署在K8s/OpenShift, 无需配置Nginx fallback
2. 内网工具系统, SEO不重要
3. 兼容性更好, 支持老旧浏览器

8.3 性能优化

Q7: 如何优化大量树节点的渲染性能?

答:

优化方案:

1. **React.memo**: 避免未变化子树重渲染
2. **useRef**: 存储不需要触发重渲染的数据
3. **useDeferredValue**: 延迟更新, 避免编辑卡顿
4. **虚拟滚动**: 大量节点时使用 `react-window`
5. **懒加载**: 子节点按需加载

本项目应用:

```
// 1. memo优化
const RightTree = memo(({ data, ...props }) => {
  // ...
}, (oldProps, newProps) =>
  JSON.stringify(oldProps.data) === JSON.stringify(newProps.data)
);

// 2. useRef穿透
const eventRef = useRef<DataSourceType[]>([]);

// 3. useDeferredValue
const deferredQuery = useDeferredValue(tree);
```

Q8: 如何设计一个高性能的表单校验系统?

答:

设计思路:

1. **防抖校验**: 500ms防抖, 避免频繁校验
2. **递归遍历**: 遍历所有节点收集校验失败的Key
3. **DOM高亮**: 直接操作DOM设置错误边框
4. **搜索导航**: AutoComplete快速定位错误字段

本项目实现:

```
// 1. 防抖校验
const { run: debouncedValidate } = useDebounceFn(
  (data: treeItem[]) => {
    const unvalidateKeys = validateAllKeys(data);
    setCardBorderColor(unvalidateKeys); // DOM高亮
  },
  { wait: 500 }
);

// 2. 递归校验
export const validateAllKeys = (data: treeItem[]) => {
  let keys: string[] = [];
  data?.forEach((item) => {
    if (!item?.children && item.require && !isEmpty(item?.value)) {
      keys.push(item?.key);
    }
    if (item?.children) {
      keys = keys.concat(validateAllKeys(item?.children));
    }
  });
  return keys;
};
```

8.4 工程化

Q9: 如何配置Monaco Editor支持YAML智能提示?

答:

```
// 1. 安装依赖
// @monaco-editor/react, monaco-editor, monaco-yaml

// 2. 配置webpack
// config/config.ts
plugins: [new MonacoWebpackPlugin({ languages: ['yaml'] })]

// 3. 配置YAML Schema
import * as monaco from 'monaco-editor';
import { configureMonacoYaml } from 'monaco-yaml';

configureMonacoYaml(monaco, {
  schemas: [{
    uri: 'k8s://schema',
    fileMatch: ['*'],
    schema: k8sSchema
  }]
});
```

Q10: 如何设计一个统一的错误处理体系?

答:

五级错误处理:

```
enum ErrorShowType {
  SILENT = 0,          // 静默
  WARN_MESSAGE = 1,    // 警告弹窗
  ERROR_MESSAGE = 2,    // 错误弹窗
  NOTIFICATION = 3,     // 通知框
  REDIRECT = 9,         // 重定向
}
```

错误分流:

1. BizError → 按showType选择UI展示
2. HTTP错误 → 显示状态码
3. 网络错误 → 提示重试

拦截器预留:

```
requestInterceptors: [(config) => {  
  // Token注入、请求签名等  
  return config;  
}],  
  
responseInterceptors: [(response) => {  
  // 响应数据转换、错误码处理  
  return response;  
}],
```

九、项目数据统计

9.1 代码规模

类型	数量	说明
页面组件	4个	TCList、TesterList、FgcgenList、EventMap
共享组件	7个	Editor、JsonEditor、YAMLEditor等
业务子组件	12个	TCModel下的所有子组件
最大组件	628行	EditTable.tsx
API接口	30+	覆盖所有业务场景
Mock文件	14个	开发环境独立运行
工具函数	7个	deepClone、getType等

9.2 依赖统计

类别	数量	代表依赖
核心框架	3	React、UmijS、Ant Design
编辑器	3	Ace、Monaco、vanilla-jsoneditor
Hooks库	1	ahooks
拖拽库	2	@formkit/drag-and-drop、react-draggable
CSS-in-JS	1	antd-style
工具库	4	js-yaml、js-cookie、classnames、react-helmet-async

十、技术亮点总结

10.1 核心技术亮点

亮点	描述	技术价值
动态表单DSL	后端驱动的表单配置，零代码适配新网元	架构设计能力
树形数据引擎	完整的递归CRUD操作库	算法设计能力
流式日志传输	ReadableStream实时读取	异步编程能力
三模式编辑器	JSON/Events/Diff无缝切换	组件设计能力
React 18实战	startTransition、useDeferredValue	新技术跟进能力

10.2 工程化亮点

亮点	描述	工程价值
Presenter/Logic分离	业务逻辑与UI组件分离	可维护性
统一错误处理	五级错误处理体系	用户体验
多环境配置	dev/test/pre/production四套环境	部署灵活性
CI/CD流水线	GitLab CI自动化	开发效率
代码质量保障	ESLint+Prettier+Husky+TypeScript	代码质量

10.3 面试讲述建议

重点讲述:

- 1. 动态表单渲染引擎 - 体现架构设计能力
- 2. 树形数据操作引擎 - 体现算法设计能力
- 3. React 18新特性应用 - 体现新技术跟进能力
- 4. 性能优化策略 - 体现优化意识

讲述框架:

1. 背景：为什么需要这个系统
2. 难点：技术挑战是什么
3. 方案：如何设计解决
4. 实现：关键代码实现
5. 优化：如何性能优化
6. 收获：学到了什么

十一、源码级深度解析

11.1 组件通信模式详解

父子组件通信 (Props Down)

```
TCList/index.tsx (父)
|
|   └─> TCModal (子)
|       └─ currentRow: TCCase           // 当前行数据
|       └─ mode: 'Add'|'Edit'|'Copy'    // 操作模式
|       └─ createModalOpen: boolean     // 打开状态
|       └─ onClose: () => void           // 关闭回调
|
|   └─> RightTree (子)
|       └─ data: TreeData[]             // 树数据
|       └─ groupId: string               // 当前分组
|       └─ onSelect: (key) => void      // 选中回调
|
|   └─> TCSummary (子)
|       └─ groupId: string
|       └─ tcId: string
|       └─ ns: string
```

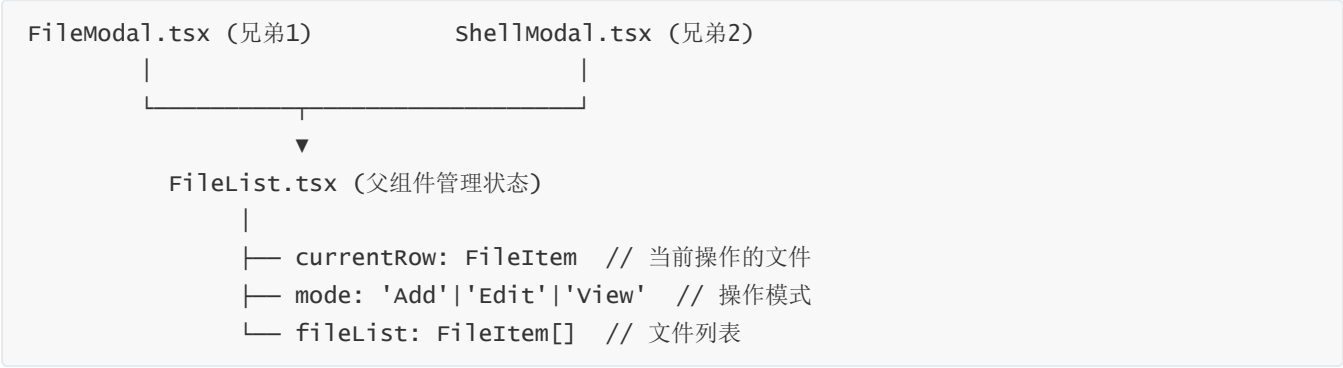
跨层组件通信 (useRef穿透模式)

```
Events.tsx
|
|   └─ eventRef = useRef<DataSourceType[]>([]) // 跨组件共享的可变数据源
|       |
|       |   └─> EditTable.tsx (读写eventRef)
|       |       └─ eventRef.current = newData // 直接修改，不触发重渲染
|       |
|       |   └─> AddEvent.tsx (读写eventRef)
|       |       └─ eventRef.current = [...eventRef.current, newEvent]
|       |
|   └─ setEventLists(eventRef.current) // 折叠时同步到state触发重渲染
```

设计意图:

- `eventRef` 作为"真相来源", 避免频繁触发父组件重渲染
- 折叠/展开时才同步到 `state`, 减少不必要的渲染
- 这是React中"状态提升"的变体: 将数据提升到父组件, 但通过ref避免重渲染

兄弟组件通信（状态提升）

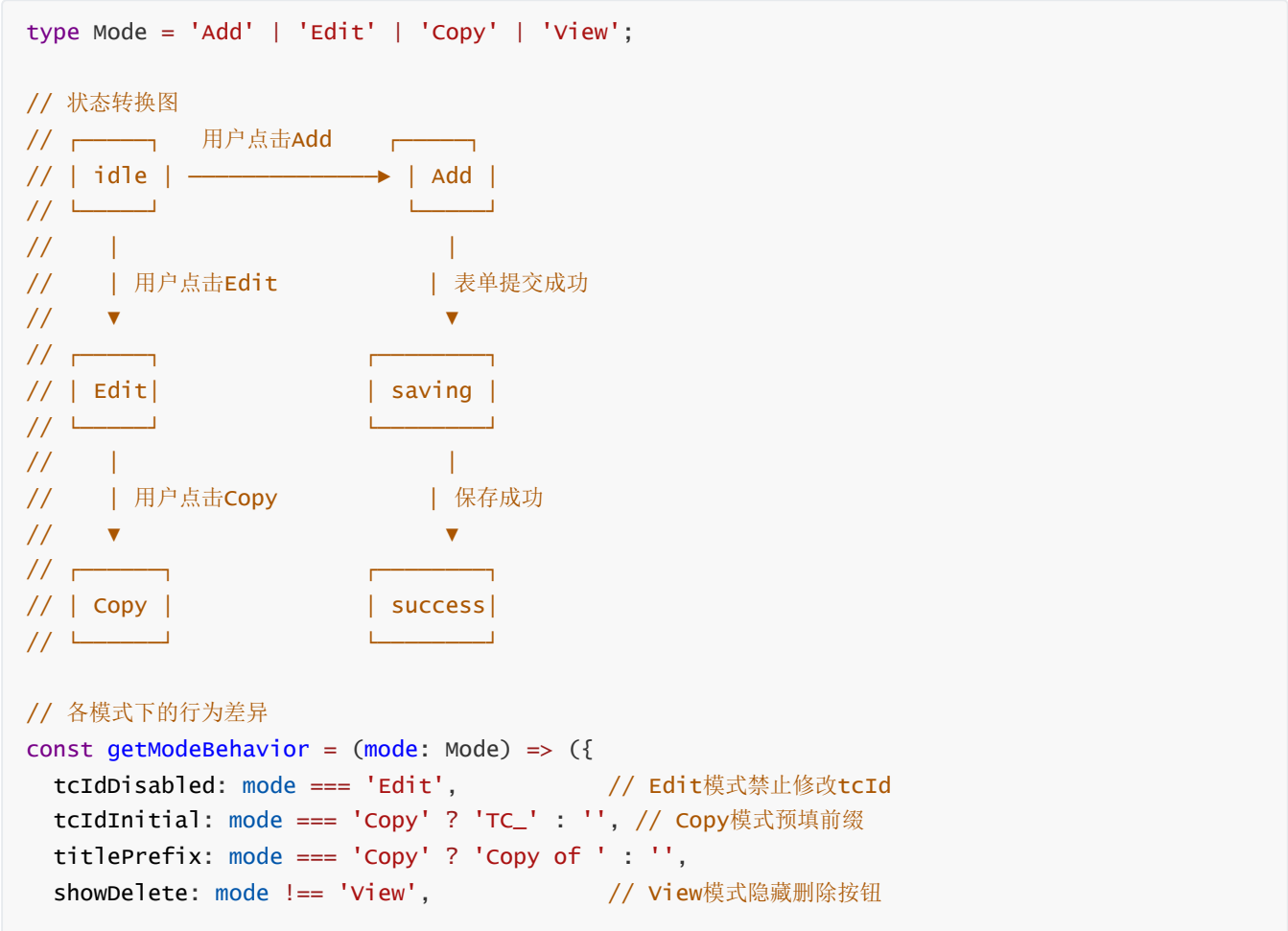


跨模块通信（Context + Model）



11.2 状态机模式详解

模式状态机（Mode State Machine）



```
showRun: mode === 'view', // view模式显示运行按钮
});
```

轮询状态机 (Polling State Machine)

```
// 轮询状态: 0 = 停止, 500 = 运行中
type PollingState = 0 | 500;

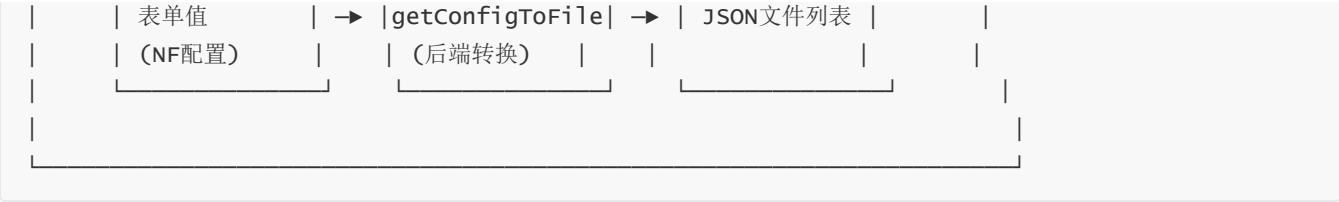
const pollingReducer = (state: PollingState, action: 'TOGGLE'): PollingState => {
  return state === 0 ? 500 : 0;
};

// 使用
const [polling, dispatch] = useReducer(pollingReducer, 500);
<Button onClick={() => dispatch('TOGGLE')}>
  {polling ? '停止轮询' : '开启轮询'}
</Button>
```

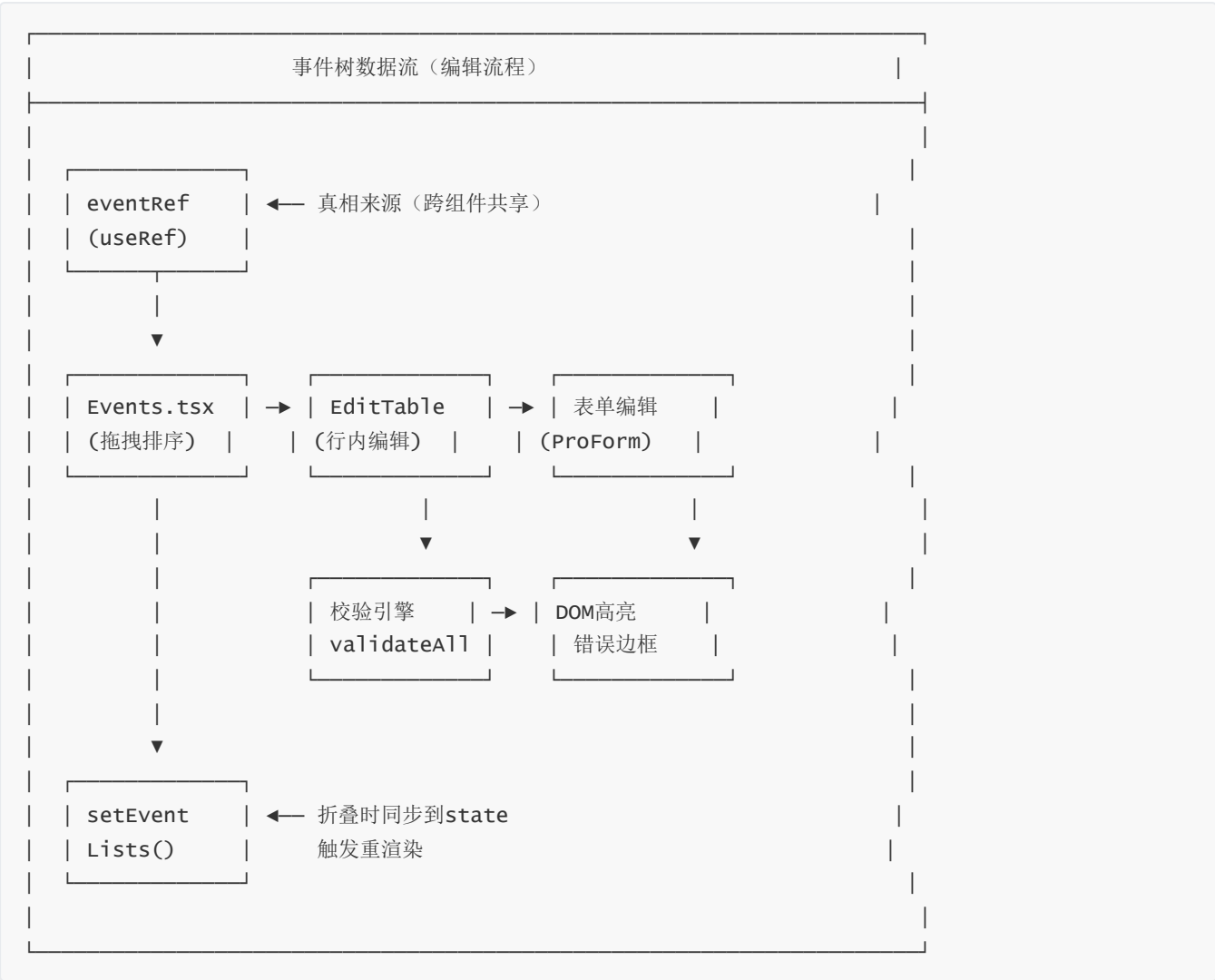
11.3 数据流详解

TC配置数据流 (完整流程)





事件树数据流（编辑流程）



11.4 设计模式深度分析

策略模式（Strategy Pattern）

应用位置: `renderTreeItem` 函数 (EditTable.tsx)

```
// 策略接口
interface RenderStrategy {
  type: string;
  render: (record: treeItem, mode: Mode) => React.ReactNode;
}

// 具体策略
const BoolStrategy: RenderStrategy = {
  type: 'bool',
```

```

render: (record, mode) => (
  <ProFormSelect
    disabled={mode === 'view'}
    rules={[{ required: !!record.require }]}
    request={async () => [
      { label: 'true', value: true },
      { label: 'false', value: false },
    ]}
  />
),
};

const DigitStrategy: RenderStrategy = {
  type: 'digit',
  render: (record, mode) => (
    <ProFormDigit
      disabled={mode === 'view'}
      fieldProps={{
        precision: 0,
        max: Number.MAX_SAFE_INTEGER,
        min: Number.MIN_SAFE_INTEGER,
      }}
    />
  ),
};

// 策略注册表
const strategyMap = new Map<string, RenderStrategy>();
strategyMap.set('bool', BoolStrategy);
strategyMap.set('digit', DigitStrategy);
strategyMap.set('text', TextStrategy);
strategyMap.set('float', FloatStrategy);
strategyMap.set('select', SelectStrategy);
strategyMap.set('digitArray', DigitArrayStrategy);
strategyMap.set('floatArray', FloatArrayStrategy);
strategyMap.set('textArray', TextArrayStrategy);

// 使用策略
const renderTreeItem = (record: treeItem, mode: Mode) => {
  const strategy = strategyMap.get(record.type);
  return strategy ? strategy.render(record, mode) : null;
};

```

观察者模式 (Observer Pattern)

应用位置: `Form.usewatch` + `useDebounceEffect`

```

// 观察者: 监听表单值变化
const values = ProForm.usewatch([], form);

// 被观察者: 表单值变化时触发校验
useEffect(() => {
  form.validateFields({ recursive: true })

```

```

    .then(() => {})
    .catch((errorInfo) => {
      // 校验失败时高亮错误字段
      errorInfo.errorFields.forEach((item) =>
        setCardBorderColor(item.name.join('_'), 'red')
      );
    });
  }, [form, values]); // 依赖values，每次变化都触发

```

中介者模式 (Mediator Pattern)

应用位置: 父组件协调多个子组件

```

// 中介者: TcList/index.tsx
const TcList = () => {
  const [currentRow, setCurrentRow] = useState<TCCase>();
  const [mode, setMode] = useState<Mode>('Add');
  const [createModalOpen, setCreateModalOpen] = useState(false);
  const actionRef = useRef<ActionType>();

  // 协调函数：子组件通过回调通知中介者
  const handleAdd = () => {
    setMode('Add');
    setCurrentRow(undefined);
    setCreateModalOpen(true);
  };

  const handleEdit = (record: TCCase) => {
    setMode('Edit');
    setCurrentRow(record);
    setCreateModalOpen(true);
  };

  const handleSuccess = () => {
    setCreateModalOpen(false);
    actionRef.current?.reload(); // 通知表格刷新
  };

  // 传递协调函数给子组件
  return (
    <>
      <ProTable actionRef={actionRef} />
      <TCModal
        mode={mode}
        currentRow={currentRow}
        open={createModalOpen}
        onSuccess={handleSuccess}
      />
    </>
  );
};

```

建造者模式 (Builder Pattern)

应用位置: `getInitValue` 数据转换

```
// 建造者: 分步构建复杂对象
class FormConfigBuilder {
    private formConfiguration = {};
    private pathMapToData: string[] = [];

    // 步骤1: 构建NF配置映射
    buildNFMapping(formJson: any): this {
        if (formJson?.children?.length) {
            formJson.children.forEach((item: any) => {
                this.formConfiguration = {
                    ...this.formConfiguration,
                    [item.key]: item.children[0],
                };
            });
        }
        return this;
    }

    // 步骤2: 递归提取路径映射
    buildPathMapping(): this {
        ['amf', 'udm', 'pcf', 'chf', 'nrf', 'upf', 'sgw'].forEach((nf) => {
            this.traverseNFConfig(this.formConfiguration[nf]);
        });
        return this;
    }

    // 步骤3: 递归遍历
    private traverseNFConfig(obj: any): void {
        if (Array.isArray(obj)) {
            obj.forEach((item) => this.traverseNFConfig(item));
        } else if (typeof obj === 'object' && obj !== null) {
            if (obj?.controlType === 'form' && obj?.path !== '') {
                Object.keys(obj.data || {}).forEach((key) => {
                    this.pathMapToData.push(`${obj.path}.${key}`);
                });
            }
            Object.keys(obj).forEach((key) => {
                if (key === 'children' && Array.isArray(obj[key])) {
                    this.traverseNFConfig(obj[key]);
                }
            });
        }
    }

    // 构建结果
    build() {
        return {
            formConfiguration: this.formConfiguration,
            pathMapToData: this.pathMapToData,
        };
    }
}
```

```
    };  
  }  
}  
  
// 使用  
const result = new FormConfigBuilder()  
  .buildNFMapping(formJson)  
  .buildPathMapping()  
  .build();
```

十二、边界情况处理详解

12.1 空值防御

```
// 1. 可选链 + 空值合并  
const value = item?.children?.length ?? 0;  
  
// 2. 数组空值防御  
data?.forEach((item) => { ... }); // 使用可选链  
  
// 3. 对象空值防御  
if (!parentObj[key]) parentObj[key] = {}; // 自动创建空对象  
  
// 4. 参数空值防御  
const findItemByKey = (data: any, targetKey: string | undefined) => {  
  if (!targetKey) return null; // 提前返回  
  // ...  
};
```

12.2 类型安全

```
// 1. 类型断言（谨慎使用）  
const error = err as Error;  
  
// 2. 类型守卫  
const isError = (error: unknown): error is Error => {  
  return error instanceof Error;  
};  
  
// 3. 枚举替代字符串  
type Mode = 'Add' | 'Edit' | 'Copy' | 'View';  
const isValidMode = (mode: string): mode is Mode => {  
  return ['Add', 'Edit', 'Copy', 'View'].includes(mode);  
};
```

12.3 异步边界

```
// 1. AbortController取消请求
const abortController = new AbortController();
fetch(url, { signal: abortController.signal });
// 清理
useEffect(() => {
  return () => abortController.abort();
}, []);

// 2. 竞态条件处理
const [data, setData] = useState(null);
const [loading, setLoading] = useState(false);

const fetchData = async () => {
  setLoading(true);
  const result = await api.getData();
  if (mounted) { // 检查组件是否已挂载
    setData(result);
    setLoading(false);
  }
};

// 3. 错误边界
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError(error: Error) {
    return { hasError: true };
  }

  componentDidCatch(error: Error, errorInfo: React.ErrorInfo) {
    console.error('ErrorBoundary caught:', error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

12.4 性能边界

```
// 1. 大数据量防抖
const { run: debouncedSearch } = useDebounceFn(
  (value: string) => setSearchResult(filterOptions(value)),
  { wait: 300 }
);
```

```
// 2. 虚拟滚动（大量列表项）
import { FixedSizeList } from 'react-window';

<FixedSizeList
  height={400}
  itemCount={items.length}
  itemSize={35}
  width={300}
>
  ({ index, style }) => (
    <div style={style}>{items[index].name}</div>
  )
</FixedSizeList>

// 3. Web worker（复杂计算）
const worker = new Worker(new URL('./worker.js', import.meta.url));
worker.postMessage({ type: 'SORT', data: largeArray });
worker.onmessage = (e) => setSortedData(e.data);
```

十三、面试深度问答（高级篇）

13.1 React并发模式

Q11: React 18的并发模式是什么？项目中如何应用？

答:

并发模式核心:

- React可以同时准备多个版本的UI
- 高优先级更新（用户输入）可以打断低优先级更新（数据加载）
- 通过 `startTransition` 和 `useDeferredValue` 标记低优先级更新

本项目应用:

```
// 1. startTransition - 标记非紧急更新
const handleSave = (record: treeItem, value: any) => {
  // 先同步更新ref（紧急）
  eventRef.current = eventRef.current?.map((item) =>
    item.key === data.key ? { ...data, tree: recordList } : item
  );
  // 再异步更新state（非紧急）
  startTransition(() => setRecordList(recordList));
};

// 2. useDeferredValue - 延迟渲染
const tree = useMemo(() => recordList, [JSON.stringify(recordList)]);
const deferredQuery = useDeferredValue(tree);
// 使用deferredQuery渲染表格，优先级低于用户输入
<EditableProTable value={deferredQuery} />
```

优势:

- 编辑表格时不会卡顿
- 模式切换更流畅
- 大数据量更新不阻塞交互

Q12: useMemo和useCallback的区别？ 什么时候用哪个？

答:

Hook	用途	缓存内容	依赖
useMemo	缓存计算结果	返回值	计算依赖
useCallback	缓存函数引用	函数本身	函数依赖

本项目应用:

```
// useMemo - 缓存计算结果
const flatTreeData = useMemo(() => {
  return flattenTree(treeData); // 昂贵的计算
}, [treeData]);

// useCallback - 缓存函数引用
const handleSelect = useCallback((key: string) => {
  setSelectedKey(key);
  navigate(`/tc/${key}`);
}, [navigate]);

// 传递给memo组件时必须用useCallback
const RightTree = memo(({ onSelect, ...props }) => {
  // ...
});

// 正确：函数引用稳定，不会触发重渲染
<RightTree onSelect={handleSelect} />

// 错误：每次渲染创建新函数，触发重渲染
<RightTree onSelect={(key) => setSelectedKey(key)} />
```

13.2 架构设计

Q13: 如何设计一个可扩展的组件库？

答:

设计原则:

1. 单一职责: 每个组件只做一件事
2. 开闭原则: 对扩展开放，对修改关闭

3. 依赖倒置: 依赖抽象而非具体实现

本项目组件设计:

```
// 1. 基础组件（无业务逻辑）
const Button = ({ children, ...props }) => <button {...props}>{children}</button>;

// 2. 业务组件（封装业务逻辑）
const DeleteButton = ({ onConfirm, children }) => (
  <Popconfirm title="确认删除?" onConfirm={onConfirm}>
    <Button danger>{children}</Button>
  </Popconfirm>
);

// 3. 容器组件（管理状态）
const DeleteButtonContainer = ({ id, onDelete }) => {
  const handleConfirm = async () => {
    await onDelete(id);
    message.success('删除成功');
  };
  return <DeleteButton onConfirm={handleConfirm}>删除</DeleteButton>;
};

// 4. 复合组件（组合基础组件）
const UserCard = ({ user }) => (
  <Card>
    <Avatar src={user.avatar} />
    <span>{user.name}</span>
    <DeleteButtonContainer id={user.id} onDelete={deleteUser} />
  </Card>
);
```

Q14: 如何处理大型项目的状态管理?

答:

状态分层策略:



- 分页状态
- 3. 组件状态（组件内部）
 - ↳ `useState` / `useRef`
 - 展开状态
 - 输入值
 - 校验状态
- 4. 持久化状态（跨会话）
 - ↳ `localStorage` / `sessionStorage` / `Cookies`
 - 用户偏好
 - Token
 - 展开状态

本项目实践:

```
// 1. 全局状态 - UmiJS Model
// src/models/configuration.ts
export default () => {
  const [dataSource, setDataSource] = useState([]);
  return { dataSource, setDataSource };
};

// 2. 页面状态 - useState
const [currentRow, setCurrentRow] = useState<TCCase>();
const [mode, setMode] = useState<Mode>('Add');

// 3. 组件状态 - useRef（不触发重渲染）
const eventRef = useRef<DataSourceType[]>([]);

// 4. 持久化状态 - localStorage
localStorage.setItem('expandKeys', JSON.stringify(expandKeys));
const expandKeys = JSON.parse(localStorage.getItem('expandKeys') || '{}');
```

13.3 性能优化

Q15: 如何分析和优化React应用性能?

答:

性能分析工具:

1. **React DevTools Profiler**: 可视化组件渲染时间
2. **Chrome DevTools Performance**: 分析主线程活动
3. **Lighthouse**: 综合性能评估

优化策略:

```
// 1. 减少重渲染
```

```

// 使用React.memo
const MemoizedComponent = memo(({ data }) => {
  // ...
}, (prev, next) => prev.data.id === next.data.id);

// 使用useMemo缓存计算结果
const expensiveValue = useMemo(() => {
  return heavyComputation(data);
}, [data]);

// 使用useCallback缓存函数
const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);

// 2. 懒加载组件
const LazyComponent = lazy(() => import('./LazyComponent'));
<Suspense fallback={<Loading />}>
  <LazyComponent />
</Suspense>

// 3. 虚拟滚动
import { FixedSizeList } from 'react-window';
<FixedSizeList height={400} itemCount={10000} itemSize={35}>
  ({ index, style }) => <div style={style}>Item {index}</div>
</FixedSizeList>

// 4. web worker (复杂计算)
const worker = new Worker('./worker.js');
worker.postMessage({ type: 'SORT', data: largeArray });

```

Q16: 如何优化首屏加载速度?

答:

优化策略:

```

// 1. 路由懒加载
const TList = lazy(() => import('./pages/TList'));
const TesterList = lazy(() => import('./pages/TesterList'));

// 2. 代码分割
// UmiJS自动分割, 无需手动配置

// 3. 资源压缩
// config/config.ts
export default {
  esbuildMinifyIIFE: true,
  jsMinifier: 'esbuild',
  drop: ['console', 'debugger'], // 生产环境移除console
};

```

```
// 4. 图片优化
// 使用webP格式
// 懒加载图片


// 5. CDN加速
// externals配置
externals: {
  react: 'React',
  'react-dom': 'ReactDOM',
}

// 6. Service Worker缓存
// PWA配置
pwa: {
  workboxPluginMode: 'GeneratesSW',
  workboxOptions: {
    skipWaiting: true,
    clientsClaim: true,
  },
}
```

13.4 工程化

Q17: 如何设计一个可维护的代码规范?

答:

代码规范体系:

代码规范体系	
1. 代码格式化	
↳ Prettier	
- 统一缩进、引号、分号	
- 自动格式化	
2. 代码检查	
↳ ESLint	
- 未使用变量	
- 类型错误	
- 最佳实践	
3. Git Hooks	
↳ Husky + lint-staged	
- 提交前自动检查	
- 仅检查暂存区文件	
4. 提交规范	
↳ Commitlint	

- feat: 新功能
- fix: 修复bug
- docs: 文档更新

本项目配置:

```
// .eslintrc.js
{
  "extends": ["@umijs/lint"],
  "rules": {
    "no-console": "warn",
    "no-unused-vars": "error",
    "@typescript-eslint/no-explicit-any": "warn"
  }
}

// .prettierrc.js
{
  "singleQuote": true,
  "semi": true,
  "trailingComma": "all",
  "printWidth": 100
}

// .husky/pre-commit
npx lint-staged
```

Q18: 如何设计一个可扩展的API层?

答:

API层设计:

```
// 1. 基础请求封装
const request = async <T>(url: string, options?: RequestInit): Promise<T> => {
  const response = await fetch(url, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${getToken()}`,
      ...options?.headers,
    },
  });

  if (!response.ok) {
    throw new Error(`HTTP error! status: ${response.status}`);
  }

  return response.json();
}
```

```
};

// 2. API模块化
// src/services/smf-tester/api.ts
export const getTCList = (params: GetTCListParams) =>
  request<TCCase[]>(`/api/tc-list?${new URLSearchParams(params)}`);

export const addTC = (data: AddTCParams) =>
  request<TCCase>(`/api/tc`, {
    method: 'POST',
    body: JSON.stringify(data),
  });

// 3. 拦截器
const apiclient = {
  request: async <T>(config: RequestConfig): Promise<T> => {
    // 请求拦截: 注入token
    config.headers = {
      ...config.headers,
      Authorization: `Bearer ${getToken()}`,
    };

    try {
      const response = await fetch(config.url, config);
      return await response.json();
    } catch (error) {
      // 响应拦截: 统一错误处理
      if (error instanceof Error) {
        message.error(error.message);
      }
      throw error;
    }
  },
};
```

十四、项目改进方向

14.1 短期改进 (1-2周)

改进项	优先级	预期效果
消除 any 类型	高	提升类型安全
统一 waitTime 工具函数	中	减少重复代码
修复 getValueFromJson 副作用	高	消除潜在bug
完善 access.ts 权限系统	中	支持细粒度权限
移除生产环境 console.log	中	减少包体积

14.2 中期改进（1-2月）

改进项	优先级	预期效果
引入状态机库（XState）	中	简化状态管理
添加单元测试	高	提升代码质量
集成错误监控（Sentry）	中	快速定位问题
优化 deepClone 使用 structuredClone	低	简化代码
抽取NF列表为常量	低	减少硬编码

14.3 长期改进（3-6月）

改进项	优先级	预期效果
微前端架构（qiankun）	低	支持模块化部署
GraphQL替代REST	低	优化数据获取
WebSocket替代轮询	中	实时性提升
国际化（i18n）完善	低	支持多语言
性能监控集成	中	持续优化性能

十五、总结

本项目是一个5G核心网测试用例管理系统，采用React + UmijS Max + Ant Design Pro技术栈，实现了：

核心技术亮点

亮点	技术价值	难度
动态表单DSL引擎	后端驱动的表单配置，零代码适配新网元	☆☆☆
树形数据操作引擎	完整的递归CRUD操作库	☆☆☆
React 18并发特性	useDeferredValue + startTransition实战	☆☆☆
三模式文件编辑器	JSON/Events/Diff无缝切换	☆☆
流式日志传输	ReadableStream实时读取	☆☆
多编辑器集成	Ace + Monaco + vanilla-jsoneditor	☆☆
可拖拽模态框	突破Ant Design限制	☆

架构设计能力

能力	体现
分层架构	表现层 → 业务逻辑层 → 服务层 → 工具层
状态管理	全局/页面/组件/持久化四层状态策略
错误处理	五级错误处理体系 + 统一拦截器
权限系统	RBAC简化版 + 路由守卫

工程化能力

能力	体现
代码质量	ESLint + Prettier + Husky + TypeScript
CI/CD	GitLab CI自动化流水线
多环境	dev/test/pre/production四套环境
容器化	Docker + K8s部署

面试讲述建议

核心故事线:

背景：5G核心网测试痛点
↓
难点：复杂配置、动态表单、实时日志
↓
方案：动态表单DSL、树形数据引擎、流式传输
↓
实现：递归渲染、React 18并发特性、多编辑器集成
↓
优化：memo/useDeferredValue/防抖/轮询控制
↓
收获：架构设计、算法思维、性能优化

重点讲述:

- 1. 动态表单渲染引擎 - 体现架构设计能力
- 2. 树形数据操作引擎 - 体现算法设计能力
- 3. React 18新特性应用 - 体现新技术跟进能力
- 4. 性能优化策略 - 体现优化意识
- 5. 工程化实践 - 体现团队协作能力